Quick Sort

Resumo completo — baseado em `Aula6_AnaliseOrdenacao.pdf` e `Lista6_AnaliseOrdenacao.pdf` (Exercícios 11 e 12; partes dos Exercícios 3 e 15). Usa a teoria de 01 — Recursividade e Recorrências.

Propriedade	Quick Sort clássico (in-place, Lomuto)
Estratégia	**dividir para conquistar**: escolhe um **pivô**, **particiona** (menores à esquerda, maiores à direita) e ordena os lados recursivamente
Melhor caso	$\Theta(n \log n)$ — partições equilibradas
Caso médio	$\Theta(n \log n)$ — sob hipóteses usuais de entrada e pivô
Pior caso	$\Theta(n^2)$ — partições extremamente desequilibradas
Espaço	in-place no vetor; pilha $\Theta(\log n)$ (equilibrado) a $\Theta(n)$ (pior caso)
Estável	Não, na versão usual

---

1. Ideia

Escolhe um **pivô** e reorganiza os elementos em três grupos:

- elementos **menores** (ou iguais) que o pivô;
- o **pivô**, já na sua **posição final**;
- elementos **maiores** que o pivô.

Depois, ordena recursivamente as regiões à esquerda e à direita.

Exemplo com pivô 5: `[8, 3, 6, 2, 5, 7, 1]` → uma partição possível é `[3, 2, 1]` **5** `[8, 6, 7]`. (A disposição interna dos grupos depende da implementação.)

> **Diferença para o Merge Sort:** no Merge, divide-se em metades **sem olhar os valores** e o trabalho pesado vem **depois** (intercalação). No Quick, o trabalho pesado vem **antes** (partição em torno do pivô) e, após ordenar as partes, **não há intercalação final**.

---

2. Implementação didática (com listas auxiliares)

```
def quick_sort(v):
    if len(v) <= 1:
        return v
    pivo = v[len(v) // 2]                  # elemento central como pivô
    menores = [x for x in v if x < pivo]
    iguais  = [x for x in v if x == pivo]  # todos os iguais ao pivô juntos
    maiores = [x for x in v if x > pivo]
    return quick_sort(menores) + iguais + quick_sort(maiores)
```

- Simplifica a compreensão da partição e trata bem duplicatas (grupo `iguais`).
- **Não é in-place:** cria listas auxiliares proporcionais à entrada.

3. Implementação clássica (in-place, partição de Lomuto)

```
def quick_sort(v, inicio, fim):
    if inicio < fim:
        pos_pivo = particionar(v, inicio, fim)
        quick_sort(v, inicio, pos_pivo - 1)     # região à esquerda do pivô
        quick_sort(v, pos_pivo + 1, fim)        # região à direita do pivô
    return v

def particionar(v, inicio, fim):
    pivo = v[fim]                               # 1. pivô = último elemento da região
    i = inicio - 1                              # i = fim da zona dos "<= pivô"
    for j in range(inicio, fim):                # 2. particionamento
        if v[j] <= pivo:
            i += 1
            v[i], v[j] = v[j], v[i]
    v[i + 1], v[fim] = v[fim], v[i + 1]         # pivô vai para a posição definitiva
    return i + 1
```

Chamada inicial: `quick_sort(v, 0, len(v) - 1)`.

Como funciona:

1. **Escolha do pivô:** o último elemento da região.
2. **Particionamento:** `j` percorre a região; quando `v[j] <= pivo`, ele é levado para a zona da esquerda (posição `i`). Ao final, o pivô é trocado para `i + 1`, sua posição definitiva.
3. **Recursão:** ordena as duas regiões **sem incluir o pivô**, que já está no lugar.

Partição passo a passo em `[8, 3, 6, 2, 5, 7, 4]` (pivô = 4)

`j`	`v[j]`	`v[j] <= 4?`	Ação	`i`	Vetor
—	—	—	início	−1	`[8, 3, 6, 2, 5, 7, 4]`
0	8	não	—	−1	`[8, 3, 6, 2, 5, 7, 4]`
1	3	sim	`i = 0`, troca v[0] ↔ v[1]	0	`[3, 8, 6, 2, 5, 7, 4]`
2	6	não	—	0	`[3, 8, 6, 2, 5, 7, 4]`
3	2	sim	`i = 1`, troca v[1] ↔ v[3]	1	`[3, 2, 6, 8, 5, 7, 4]`
4	5	não	—	1	`[3, 2, 6, 8, 5, 7, 4]`
5	7	não	—	1	`[3, 2, 6, 8, 5, 7, 4]`
fim			troca v[2] ↔ v[6] (pivô)		`[3, 2, 4, 8, 5, 7, 6]`

O pivô 4 fica na posição 2 (3ª posição contando a partir de 1). Recursão:

```
quick_sort(0..6) [8, 3, 6, 2, 5, 7, 4]  pivô 4 -> [3, 2] 4 [8, 5, 7, 6]
  quick_sort(0..1) [3, 2]               pivô 2 -> [] 2 [3]
  quick_sort(3..6) [8, 5, 7, 6]         pivô 6 -> [5] 6 [7, 8]
    quick_sort(5..6) [7, 8]             pivô 8 -> [7] 8 []
Resultado: [2, 3, 4, 5, 6, 7, 8]
```

Comparação das duas versões

Característica	Quick Sort didático	Quick Sort clássico
Estratégia	cria listas de menores, iguais e maiores	particiona o vetor original
Pivô	elemento central por índice	último elemento da região
Memória auxiliar	listas adicionais ($\Theta(n)$ por nível)	não cria listas proporcionais à entrada
In-place	Não	Sim, na organização do vetor
Recursividade	duas chamadas sobre listas	duas chamadas sobre intervalos do vetor
Estabilidade	não é garantida em geral*	Não
Melhor / médio / pior	$\Theta(n \log n)$ / $\Theta(n \log n)$ / $\Theta(n^2)$	$\Theta(n \log n)$ / $\Theta(n \log n)$ / $\Theta(n^2)$

* Detalhe: como as três listas são montadas percorrendo `v` da esquerda para a direita, **este código específico** mantém a ordem relativa entre elementos de mesma chave. A versão clássica, com trocas de longa distância, não mantém.

> **Memória da versão clássica:** evita listas auxiliares, mas usa a **pilha de recursão**: tipicamente $\Theta(\log n)$ no caso equilibrado e até $\Theta(n)$ no pior caso.

---

4. Análise de complexidade passo a passo

A partição de uma região com n elementos percorre todos uma vez: $\Theta(n)$ ($n-1$ comparações). O resto depende de **onde o pivô cai**: se ele deixa q elementos à esquerda e $n-1-q$ à direita,

$$T(n) = T(q) + T(n-1-q) + cn$$

4.1 Melhor caso — partições equilibradas ($q \approx n/2$)

$$T(n) = 2T(n/2) + cn$$

Mestre: $a=2$, $b=2$, $f(n) = cn = \Theta(n^{\log_2 2})$ → Caso 2 → $\boxed{T(n) = \Theta(n \log n)}$. Altura da árvore: $\approx \log_2 n$.

4.2 Pior caso — partição extrema ($q = 0$ ou $q = n-1$)

Uma parte tem $n-1$ elementos e a outra é vazia:

$$T(n) = T(n-1) + T(0) + cn = T(n-1) + cn$$

Iteração (igual ao Exemplo 2 da aula de recursividade):

$$\begin{aligned} T(n) &= T(n-1) + cn \\ &= T(n-2) + c(n-1) + cn \\ &= T(n-3) + c(n-2) + c(n-1) + cn \\ &\vdots \\ &= T(1) + c\,(2 + 3 + \cdots + n) = T(1) + c\left(\frac{n(n+1)}{2} - 1\right) \end{aligned}$$

$$\boxed{T(n) = \Theta(n^2)}$$

Profundidade da recursão: $\approx n$ → pilha $\Theta(n)$.

Quando acontece (pivô = último elemento): vetor **já ordenado** ou em **ordem inversa** — o pivô é sempre o maior (ou o menor) da região. Também com todos os elementos iguais (na Lomuto, todos vão para o lado `<=`).

4.3 Partições desequilibradas, mas proporcionais

Mesmo uma divisão ruim **proporcional**, por exemplo 9 para 1:

$$T(n) = T(9n/10) + T(n/10) + cn$$

Pela árvore de recursão, cada nível custa no máximo cn e a profundidade é $\log_{10/9} n = \Theta(\log n)$ → ainda $\Theta(n \log n)$. O desastre só acontece quando as partições são **constantemente** extremas.

4.4 Caso médio — dedução

Hipótese: todas as posições finais do pivô são igualmente prováveis (entrada aleatória ou pivô aleatório). Seja $C(n)$ o número esperado de comparações, com $C(0) = C(1) = 0$:

$$C(n) = (n-1) + \frac{1}{n}\sum_{q=0}^{n-1}\big[C(q) + C(n-1-q)\big] = (n-1) + \frac{2}{n}\sum_{q=0}^{n-1} C(q)$$

Passo 1 — multiplicar por n: $nC(n) = n(n-1) + 2\sum_{q=0}^{n-1} C(q)$.

Passo 2 — escrever a mesma equação para $n-1$: $(n-1)C(n-1) = (n-1)(n-2) + 2\sum_{q=0}^{n-2} C(q)$.

Passo 3 — subtrair as duas (a soma quase toda se cancela):

$$nC(n) - (n-1)C(n-1) = 2(n-1) + 2C(n-1)$$
$$\Rightarrow \quad nC(n) = (n+1)C(n-1) + 2(n-1)$$

Passo 4 — dividir por $n(n+1)$:

$$\frac{C(n)}{n+1} = \frac{C(n-1)}{n} + \frac{2(n-1)}{n(n+1)}, \qquad \frac{2(n-1)}{n(n+1)} = \frac{4}{n+1} - \frac{2}{n}$$

Passo 5 — "telescopar" (somar de 1 até n), com $H_n = 1 + \frac{1}{2} + \cdots + \frac{1}{n} \approx \ln n$:

$$\frac{C(n)}{n+1} = \sum_{k=1}^{n}\left(\frac{4}{k+1} - \frac{2}{k}\right) = 2H_n + \frac{4}{n+1} - 4$$

$$C(n) = 2(n+1)H_n - 4n \approx 2n\ln n \approx 1{,}39\, n\log_2 n \quad \Rightarrow \quad \boxed{C(n) = \Theta(n\log n)}$$

Conferência: $C(2) = 2 \cdot 3 \cdot 1{,}5 - 8 = 1$ ✓ (uma comparação para 2 elementos).

> **Escolha do pivô:** pivô aleatório ou "mediana de três" reduzem a probabilidade de partições muito ruins, mas **não eliminam** o pior caso teórico $\Theta(n^2)$ do Quick Sort clássico.

4.5 Espaço e estabilidade

- **Pilha:** profundidade $\approx \log_2 n$ (equilibrado) até $\approx n$ (pior caso). Truque conhecido: recursão primeiro no lado **menor** (e laço no maior) limita a pilha a $O(\log n)$.
- **Não estável:** as trocas de longa distância podem inverter iguais. Ex.: `[2a, 2b, 1]` com pivô 1: a troca final v[0] ↔ v[2] dá `[1, 2b, 2a]`.

---

5. Exercícios resolvidos

Exercício 11 — Lomuto em `v = [44, 18, 32, 7, 26, 39, 12, 48, 21, 5]`

a) Primeira partição (região 0–9, pivô = `v[9]` = **5**):

`j`	`v[j]`	`v[j] <= 5 ?`	`i`
0…8	44, 18, 32, 7, 26, 39, 12, 48, 21	todos **não**	permanece −1
fim		troca v[0] ↔ v[9]	

Vetor resultante: `[5, 18, 32, 7, 26, 39, 12, 48, 21, 44]`.

b) O pivô 5 (o menor de todos) fica na **posição 0**.

c) Subproblemas: esquerda `quick_sort(0, -1)` = **vazia**; direita `quick_sort(1, 9)` = `[18, 32, 7, 26, 39, 12, 48, 21, 44]` (partição extrema: 0 e $n-1$).

d) Continuação até ordenar:

Partição de 1–9, pivô 44:

`j`	`v[j]`	`<= 44 ?`	Ação	`i`
1–6	18, 32, 7, 26, 39, 12	sim	`i` avança (trocas consigo mesmo)	1 → 6
7	48	não	—	6
8	21	sim	`i = 7`, troca v[7] ↔ v[8]	7
fim			troca v[8] ↔ v[9]	

→ `[5, 18, 32, 7, 26, 39, 12, 21, 44, 48]`, pivô na pos 8. Subproblemas: 1–7 e 9–9.

Partição de 1–7 (`[18, 32, 7, 26, 39, 12, 21]`), pivô 21:

`j`	`v[j]`	`<= 21 ?`	Ação	`i`	Vetor (posições 1–7)
1	18	sim	`i = 1` (troca consigo)	1	`18 32 7 26 39 12 21`
2	32	não	—	1	`18 32 7 26 39 12 21`
3	7	sim	`i = 2`, troca v[2] ↔ v[3]	2	`18 7 32 26 39 12 21`
4	26	não	—	2	
5	39	não	—	2	
6	12	sim	`i = 3`, troca v[3] ↔ v[6]	3	`18 7 12 26 39 32 21`
fim			troca v[4] ↔ v[7]		`18 7 12` **`21`** `39 32 26`

→ `[5, 18, 7, 12, 21, 39, 32, 26, 44, 48]`, pivô na pos 4. Subproblemas: 1–3 e 5–7.

Demais partições:

Chamada	Região	Pivô	Resultado da partição	Vetor completo
`qs(1,3)`	`[18, 7, 12]`	12	`[7]` **12** `[18]` (pos 2)	`[5, 7, 12, 18, 21, 39, 32, 26, 44, 48]`
`qs(5,7)`	`[39, 32, 26]`	26	`[]` **26** `[32, 39]` (pos 5)	`[5, 7, 12, 18, 21, 26, 32, 39, 44, 48]`
`qs(6,7)`	`[32, 39]`	39	`[32]` **39** `[]` (pos 7)	`[5, 7, 12, 18, 21, 26, 32, 39, 44, 48]`

Vetor final: `[5, 7, 12, 18, 21, 26, 32, 39, 44, 48]`.

e) Árvore de chamadas (13 chamadas; ∅ = região vazia):

```
qs(0,9) pivô 5
├── qs(0,-1) ∅
└── qs(1,9) pivô 44
    ├── qs(1,7) pivô 21
    │   ├── qs(1,3) pivô 12
    │   │   ├── qs(1,1) [7]
    │   │   └── qs(3,3) [18]
    │   └── qs(5,7) pivô 26
    │       ├── qs(5,4) ∅
    │       └── qs(6,7) pivô 39
    │           ├── qs(6,6) [32]
    │           └── qs(8,7) ∅
    └── qs(9,9) [48]
```

f) Escolha do pivô: se o pivô cai perto da **mediana** da região, as partes têm tamanhos parecidos (árvore baixa, $\approx \log n$ níveis). Se cai perto do **mínimo ou máximo** — como 5 e 44 aqui —, uma parte fica vazia ou quase, e a árvore fica alta (até n níveis). Pivô aleatório ou mediana de três tornam o desequilíbrio constante improvável.

g) Quick Sort clássico: melhor $\Theta(n \log n)$, médio $\Theta(n \log n)$, pior $\Theta(n^2)$.

Exercício 12 — recorrências e profundidade

a) Recorrências (particionamento linear cn):

- Situação A (equilibrada): $T(n) = 2T(n/2) + cn$.
- Situação B (extrema): $T(n) = T(n-1) + T(0) + cn = T(n-1) + cn$.

b) Resolvendo A: Mestre Caso 2 (ou árvore: $\log_2 n$ níveis de custo cn) → $\Theta(n \log n)$ (seção 4.1).

c) Expandindo B até o caso base: $T(n) = T(1) + c(2 + 3 + \cdots + n) = T(1) + c\left(\frac{n(n+1)}{2} - 1\right)$ (seção 4.2).

d) A: $\Theta(n \log n)$. B: $\Theta(n^2)$.

e) Profundidade: A $\approx \log_2 n$ (pilha $\Theta(\log n)$); B $\approx n$ (pilha $\Theta(n)$).

f) Médio $n \log n$, pior n^2: o pior caso exige partições extremas em **todos** os níveis, o que é raro numa entrada aleatória; em média o pivô cai "no meio" com frequência suficiente, e até divisões proporcionais ruins (9:1) mantêm $\Theta(\log n)$ níveis. A dedução da seção 4.4 dá $\approx 1{,}39\, n \log_2 n$ comparações esperadas.

g) Relação com a aula de recursividade: a Situação A é o Exemplo 3 (resolvido por **árvore**, **Mestre** e provado por **substituição**); a Situação B é o Exemplo 2 ($T(n) = T(n-1) + n$), resolvido por **iteração**; e a diferença de profundidade é a mesma dos Praticando 8 × 9 (pilha $\Theta(n)$ × $\Theta(\log n)$).

Exercícios 3 e 15 — onde o Quick Sort aparece

- **Ex. 3c:** não oferece garantia $\Theta(n \log n)$ no pior caso por causa das partições extremas (seção 4.2) → a resposta é Heap Sort (08).
- **Ex. 15h:** comparado ao Merge Sort, perde no pior caso ($\Theta(n^2)$) e na estabilidade, mas ganha em memória (in-place). Veja 06 — Merge Sort.

Matérias correlatas

> **Recorrências:** equilibrado = $2T(n/2) + cn$ (Mestre Caso 2); extremo = $T(n-1) + cn$ (iteração, soma de Gauss); divisões diferentes $T(9n/10) + T(n/10) + cn$ (árvore, como no Praticando 3). Tudo em 01 — Recursividade.

> **Pilha de recursão** $\Theta(\log n)$ × $\Theta(n)$: seção 9 de 01.

> **Merge Sort** (mesma estratégia, garantia $n \log n$, estável, mas $\Theta(n)$ de memória): 06 — Merge Sort.
> **Heap Sort** (garantia $n \log n$ e in-place): 08 — Heap Sort.